

Bocconi

COMPUTER SCIENCE

code 30424 a.y. 2024-2025

Lesson 13

Variables and data types



Università
Bocconi
MILANO

Objectives of the lesson

- Typing the first instructions in Python
- Understanding variables and the different existing data types
- Creating the first scripts

Built-in functions

- These are the “default” functions, existing in Python standard installation (i.e. `help`, `print`, `input`, ...)
- The standard syntax of a function in Python provides the name and arguments in parentheses, separated by commas when they are more than one: `fname (arg1, arg2, ...)`

Python standard library

- Many others functions are available in various libraries of functions that allow to expand the potential of Python
- The standard library is directly installed with Python and contains several useful modules
- To access the functions of a certain module it is necessary to import it. The import is valid only for the current session
- The functions of the module are called by adding the name of the library

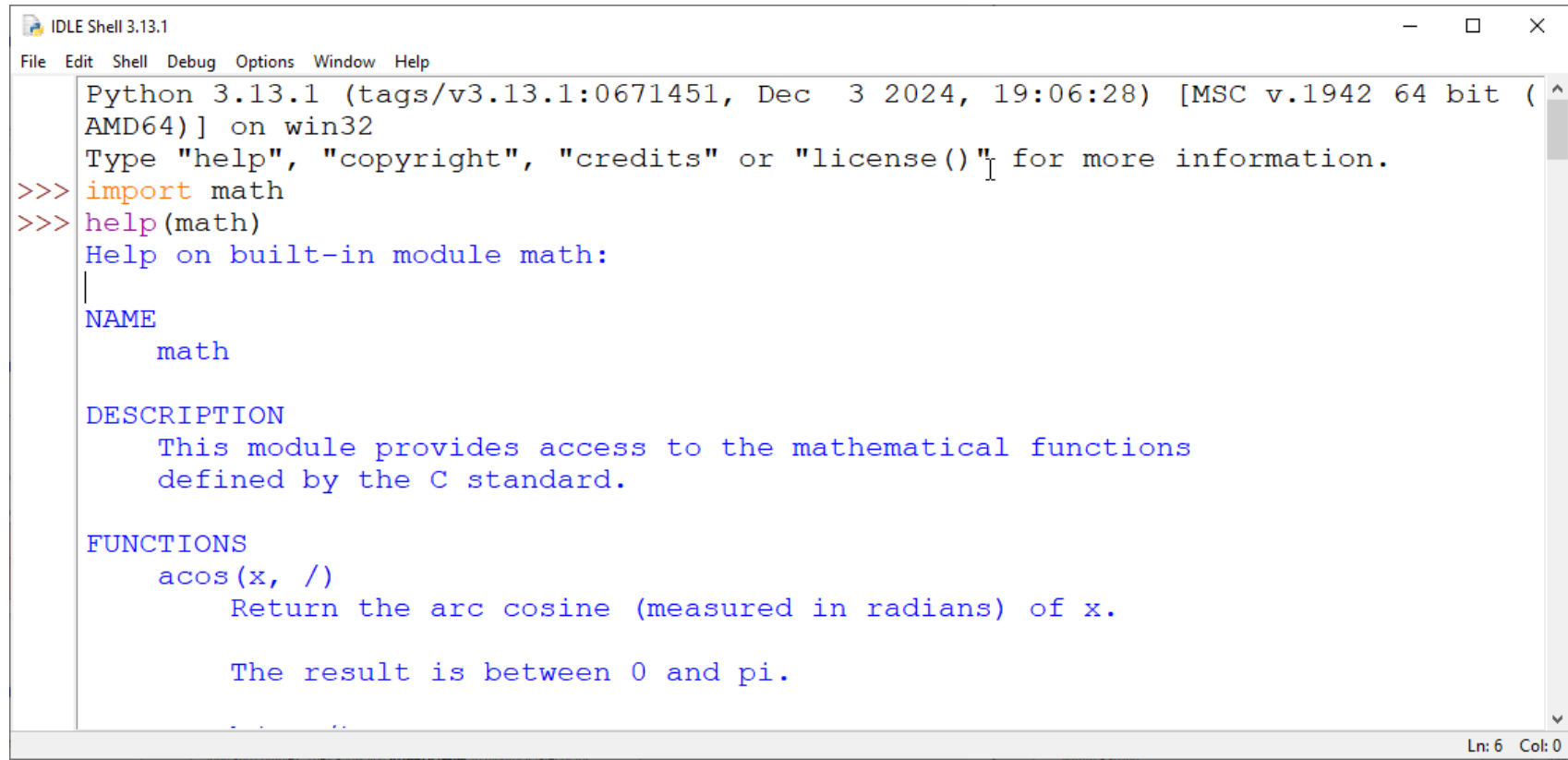
Examples:

```
>>> import math
>>> math.sqrt(16)
4.0
>>>
```

```
>>> import random
>>> random.random()
0.1315212015398296
>>>
```

help function

Shows information about a function, a data item, or a form



```
IDLE Shell 3.13.1
File Edit Shell Debug Options Window Help
Python 3.13.1 (tags/v3.13.1:0671451, Dec 3 2024, 19:06:28) [MSC v.1942 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> import math
>>> help(math)
Help on built-in module math:

NAME
    math

DESCRIPTION
    This module provides access to the mathematical functions
    defined by the C standard.

FUNCTIONS
    acos(x, /)
        Return the arc cosine (measured in radians) of x.

        The result is between 0 and pi.
```



print function

Let's start by understanding the syntax of a function and the IDLE color codes

```
*IDLE Shell 3.13.1*
File Edit Shell Debug Options Window Help
Python 3.13.1 (tags/v3.13.1:0671451 19:06:28) [MSC v.1942 64 bit
(AMD64)] on win32
Type "help", "copyright", "credits" or more information.
>>> print("Hello World!")
Hello World!
>>> print('Hello World!')
Hello World!
>>> print(Hello World!)
SyntaxError: invalid syntax. Perhaps you forgot a comma?
>>> Print'Hello World!'
SyntaxError: invalid syntax
>>> print(
    (*args, sep=' ', end='\n', file=None, flush=False)
    Prints the values to a stream, or to sys.stdout by default.
>>> print(1, 2, 3, 4, sep='*')
1*2*3*4
>>>
```

Purple for built-in functions

Green for strings

Blue for output

Red for errors

Function syntax

Use of separators

Ln: 11 Col: 6



Escape codes

- Escape codes are formed by a backslash (\) and other characters, placed within a string
- They are not displayed in the output but give specific commands
- E.g.: the code `\n` is used to wrap text, `\t` aligns to the next tab, `\` alone wraps the code

```
>>> print ('One\nTwo\nThree')
```

One

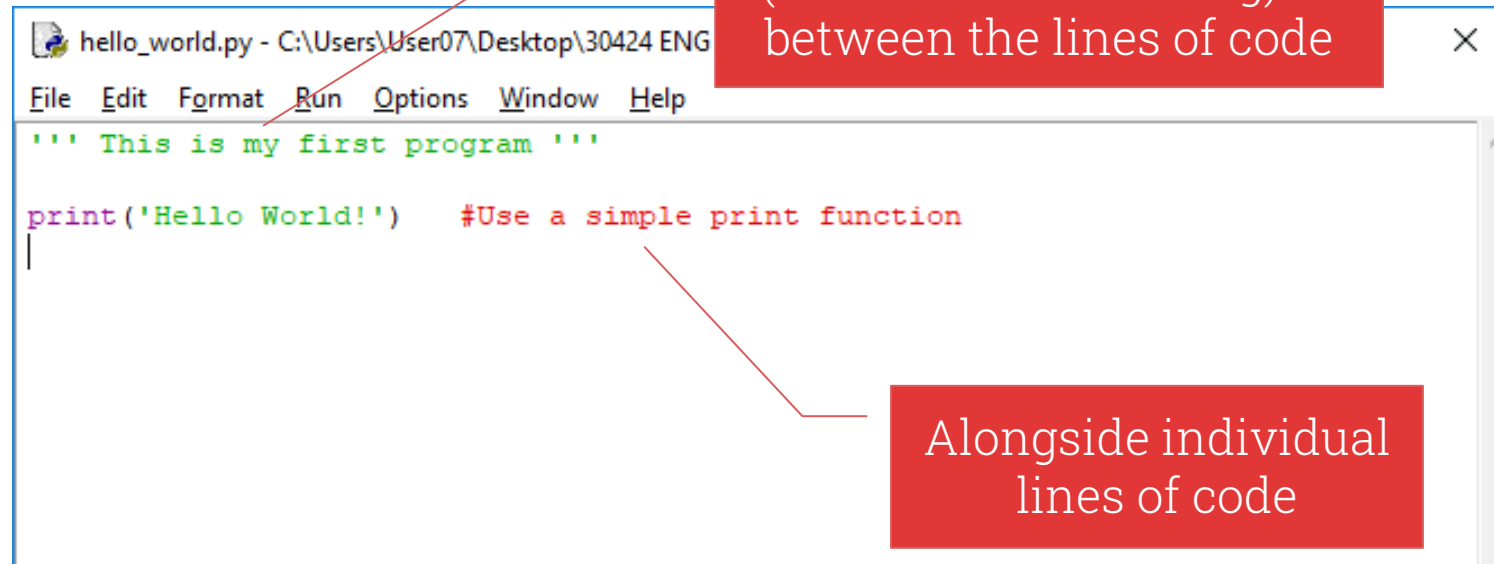
Two

Three

Commenting on the code

Comments are essential for documenting a program, especially when it is long and complex. They allow you **to read it better**, to modify it later or for debugging

Two types of comment:



The screenshot shows a Python IDE window titled 'hello_world.py - C:\Users\User07\Desktop\30424 ENG'. The menu bar includes File, Edit, Format, Run, Options, Window, and Help. The code editor contains the following text:

```
''' This is my first program '''  
  
print('Hello World!') #Use a simple print function  
|
```

Two red callout boxes provide explanations:

- A box at the top right points to the first line of code, stating: "At the beginning (documentation string) or between the lines of code".
- A box at the bottom right points to the hash symbol in the second line of code, stating: "Alongside individual lines of code".

Variables

This is a fundamental concept in all programming languages:

- A variable is **a name that represents a value** in the computer's memory
- An **assignment statement** is used to create a variable and to refer to a piece of data
- The equal sign (=) is called the **assignment operator**

Examples:

```
last_name = 'Snow'
```

```
age = 25
```

Variable names

Variable names:

- must **start with a letter or the underscore character** (_), followed by letters, numbers or underscores
- cannot contain **spaces** or **special characters**, such as @, &, \$, €, § etc.
- cannot consist of reserved Python words (**keywords**), such as **True**, **not**, **or**, **return**, **while**, **and**, **if** or function names
- would have to be **short** and **meaningful**

It is preferable not to use accented characters

Note: Python is a **case-sensitive** language, which distinguishes between variable names made up of lower and upper case characters

Assignment statements

An **assignment statement** creates a new variable and gives it a value:

```
>>> surname_professor = 'Wilson'
```

```
>>> number_students = 86
```

```
>>> g = 9.81
```

The variable name is always **on the left**, while **on the right** there must be the assigned value

The equal sign (=) is an **assignment operator**

Reassignment of variables

- Variables can refer to different values during the execution of a program
- When you assign a value to a variable, this refers to the value until a different value is assigned
- **Reassignment** is the operation of replacing one value with another

Example:

```
>>> a = 1
```

```
>>> a
```

```
1
```

```
>>> a = 2
```

```
>>> a
```

```
2
```

Assignment

Reassignment

Updating variables

One of the most common forms of reassignment is an **update**, where the new value of the variable depends on the previous one

Example:

```
>>> a = 1
>>> a
1
>>> a = a + 1
>>> a
2
```

Before updating a variable, it is essential to **initialize** it (that is assign it), as updating a non-existent variable generates an error

Multiple assignment

Python allows **multiple assignment** of variables (also called *unpacking*), which allows **acting on multiple variables directly on the same line of code**:

```
>>> a, b, z = 1, 2, 3
```

```
>>> a
```

```
1
```

```
>>> b
```

```
2
```

```
>>> z
```

```
3
```

Variables as arguments of functions

- Variables are very useful when used as function arguments
- In this case, the variable name should not be enclosed in quotes, otherwise it is interpreted as a simple string

Example:

```
>>> size = 10
>>> print (size)
10
>>> print ('size')
size
```

Data types

- When a given value is assigned to a variable, it is typed, that is, it assumes a certain type of data depending on the assigned value
- The data type determines the behavior of some operations, while others can only be performed on values of a specific data type
- To know which type of data a value belongs to, you can use the `type` function

Examples:

```
>>> type (1)
<class 'int'>
>>> type ('a')
<class 'str'>
```


Basic Python data types

Data type	Name	Description	Examples
Integer	int	Integer of arbitrary size	112, 0, -158
Real number	float	Floating-point number	2.14, -159.1234
Boolean	bool	For true or false values	True, False
String	str	Used to represent text	'Python', 'Python3', 'Web 2.0'



Reassignment to a different type

After a data of a certain type is assigned to a variable, a data of another type can be reassigned. Python has **dynamic** typing

```
>>> price = 100
```

```
>>> type (price)
```

```
>>> <class 'int'>
```

```
>>> price = 'Depends on the applied discount percentage'
```

```
>>> type (price)
```

```
>>> <class 'str'>
```



Conversion between data types

It is possible, using some specific functions, to **convert** the data type:

- `int` function: converts to an integer
- `float` function: converts to a floating-point number
- `str` function: converts to a string

Note that:

- if the conversion is impossible, an error message appears. E.g.: `int('Python')`
- `int` function does not round the fraction part, but chops it off

Example: `>>> int(9.9999)`
9

input function

- The **input** function asks the user to enter some data (i.e. an **input**)
- Once the user has typed a value and pressed the Enter key, he reads the data entered from the keyboard and returns this data to the program **as a string**
- It can be used to assign a value to a variable

Example:

```
>>> name = input("What's your name? ")
What's your name? Bob
>>> print(name)
Bob
```

Conversion from input

If we want the input function to return a data of a **different type**, we must use a **conversion function**

Example:

```
>>> value = input('Enter the desired value: ')
```

```
Enter the desired value: 127
```

```
>>> value
```

```
'127'
```

```
>>> value = value + 1
```

```
Traceback (most recent call last):
```

```
File "<pyshell#16>", line 1, in <module>
```

```
value = value + 1
```

```
TypeError: must be str, not int
```



```
>>> value = int(input('Enter the desired value: '))
```

```
Enter the desired value: 127
```

```
>>> value
```

```
127
```

```
>>> value = value + 1
```

```
>>> value
```

```
128
```



Format function

The `format` function allows number formatting, returning a string as output, using the following syntax: `format(value, format_spec)`

The `format_spec` argument is a string and can contain:

- `f` specifies that the number that you want to obtain is a **floating-point**
- `.2` indicates that we want to round off the number to the **second decimal** place
- the `,` (**comma**) adds the **thousands separator**
- the `%` symbol is used to format **in percentage**

Example:

```
>>> format(10000/7, '.2f')  
'1428.57'
```

```
>>> format(10000/7, ',.2f')  
'1,428.57'
```

```
>>> format(10000/7, ',.2%')  
'142,857.14%'
```

Shell vs. editor

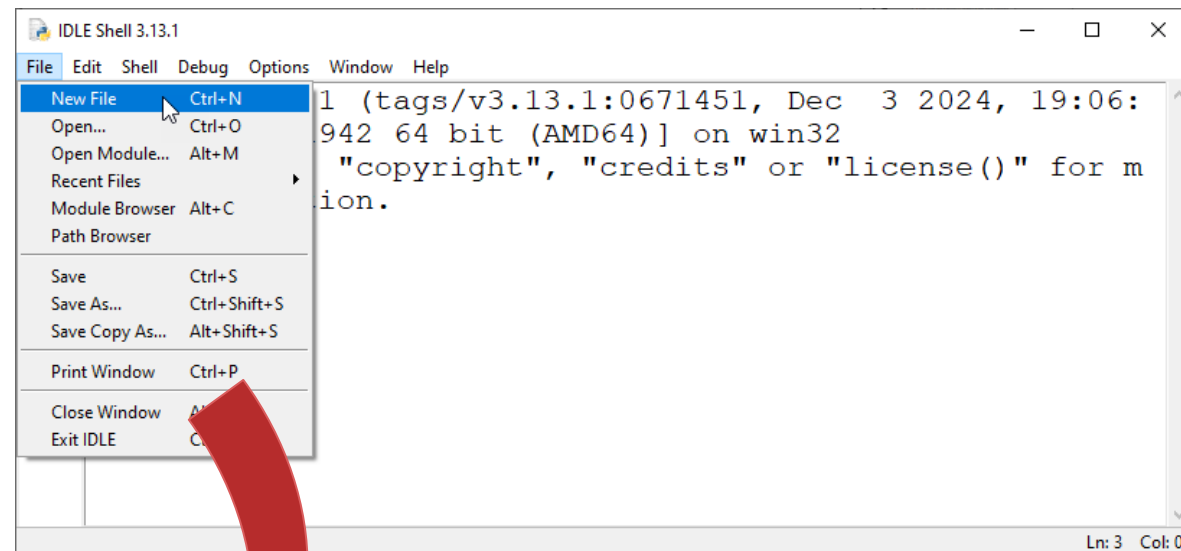
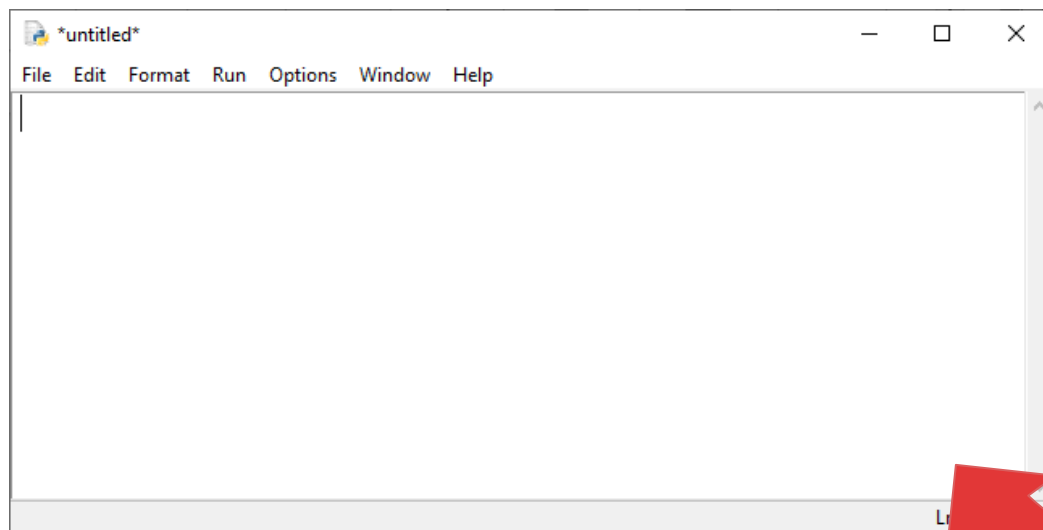
So far, to become familiar with Python we have worked in the **shell**. To create a real program, however, we need to use the **editor**:

- The **shell** is useful to start working and interacting with Python so to immediately verify the result of an operation or the effect of a command
- The **editor** allows writing a program in Python as a sequence of lines of code that must be saved as a file to be run

Writing a program

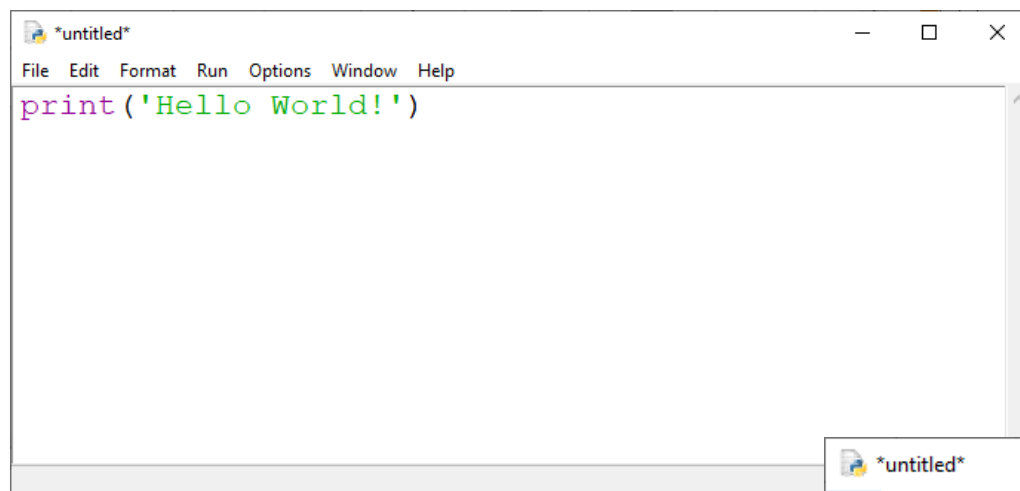
To write a new program we must open the Python **editor** from the shell menu:

File > New File



Saving the program

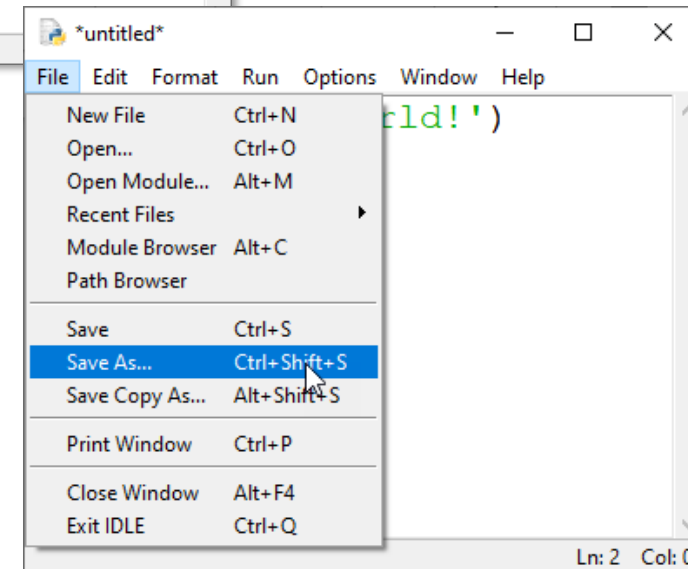
Once the code is written...



...save the program with the **File > Save As** command



Extension: filename.py



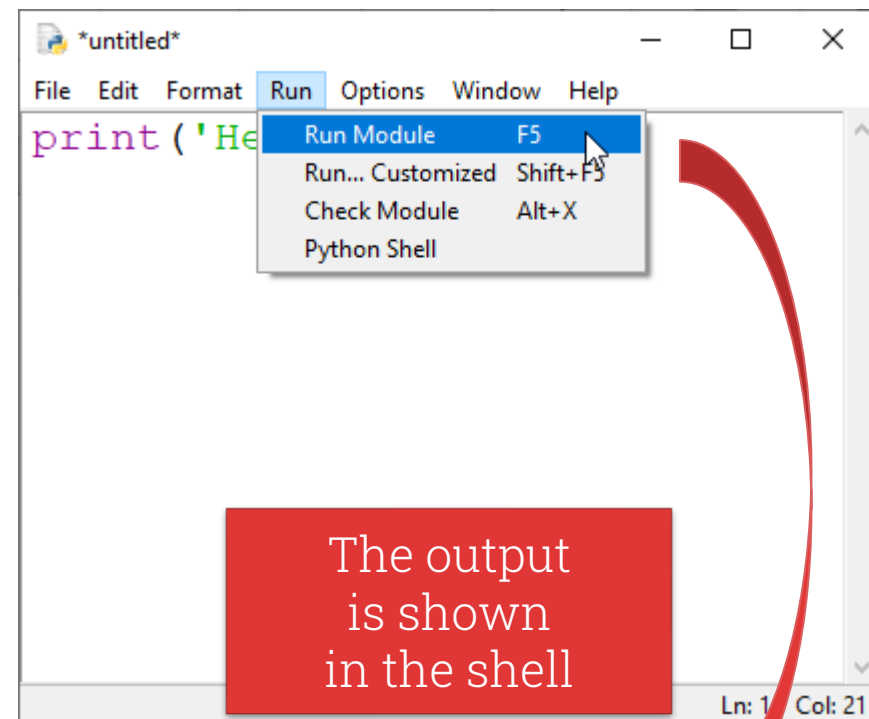
Running the program

If the file of the program is closed, to open it:

Open IDLE: File > Open...

Once the program is open in the editor, use **F5** or the **Run > Run Module** command to run it

double-click on the file does not work !
(it executes the file in the Windows shell)



The output
is shown
in the shell

Hello World!
>>> |

Exercise 1

Create a new Python file called Exercise 1.py, in which you are asked to write a program to present your own traits. You are asked to write the code necessary so that the program asks to enter name, age, height and favorite music genre.

The program must then return the following output:

I introduce myself. I am NAME and I am XX years old.

My height is XXX cm (to reach 2 meters there are XX cm missing!).

I like to listen to MUSIC_GENRE, that's my favorite.

Exercise 2

Create a new Python file called Exercise 2.py, in which you are asked to write a program that:

- asks the user for 3 integer numbers
- shows a short message with the 3 numbers on the screen
- calculates the sum and the average of the 3 numbers
- asks the user for a fourth integer number
- calculates and shows on screen the difference (in absolute value) between the average of the first 3 numbers and the fourth number entered by the user

Tools to learn with today's lecture

- Using IDLE (shell and editor)
- Understanding different data types available
- Understanding and using variables
- Using the `print`, `format`, `help`, `input`, `type` functions
- Using the escape code: `\n` , `\t` and `\`
- Using conversion functions (`str`, `int`, `float`)
- Commenting a program
- Creating, saving and running a simple program



Book references

Understanding Python:

Chapters 2, 3, 4

Python workbook:

Exercises of Unit 1